vLLM 배치 처리 및 스케줄링 최적화

개요

vLLM의 핵심 성능 우위 중 하나는 혁신적인 배치 처리 및 스케줄링 메커니즘입니다. 기존의 정적 배치 처리 방식과 달리, vLLM은 **연속 배치 처리(Continuous Batching)**와 **동적 스케줄링**을 통해 처리량을 극대화하고 지연시간을 최소화합니다. 본 문서에서는 vLLM의 스케줄러 구조와 배치 처리 알고리즘을 코드 레벨에서 상세히 분석합니다.

기존 배치 처리 방식의 한계

정적 배치 처리의 문제점

전통적인 LLM 서빙 시스템의 정적 배치 처리는 다음과 같은 근본적인 한계를 가집니다:

1. 고정 배치 크기의 비효율성

- **문제**: 시스템 시작 시 정해진 배치 크기(예: 32)로만 처리 가능
- **결과**: 요청이 적을 때도 배치가 채워질 때까지 대기하거나, 요청이 많을 때 처리 용량 제한

2. 동기화 대기로 인한 자원 낭비

- **문제**: 배치 내 가장 긴 시퀀스가 완료될 때까지 모든 요청이 대기
- **예시**: 배치에 10토큰 요청 31개와 1000토큰 요청 1개가 있으면, 31개 요청이 1000토큰 생성까지 기다림
- **결과**: GPU 자원의 90% 이상이 유휴 상태로 낭비

3. 처리량 병목 현상

- **문제**: 새로운 요청은 현재 배치가 완전히 끝날 때까지 대기

- **결과**: 평균 대기 시간 증가 및 전체 시스템 처리량 저하

```
# 기존 정적 배치 처리 방식의 의사코드
def static_batch_processing(requests, batch_size):
batches = []
for i in range(0, len(requests), batch_size):
batch = requests[i:i+batch_size]
# 배치 내 모든 시퀀스가 완료될 때까지 대기
results = process_batch_until_all_complete(batch)
batches.extend(results)
return batches
```

vLLM의 연속 배치 처리 (Continuous Batching)

핵심 개념

vLLM의 연속 배치 처리는 다음과 같은 혁신적 접근법을 사용합니다:

1. **동적 배치 구성**: 실행 중에 배치에서 완료된 시퀀스를 제거하고 새로운 시퀀스를 추가
2. **이터레이션 기반 처리**: 각 이터레이션마다 배치를 재구성
3. **자원 효율성**: GPU 자원을 최대한 활용하여 처리량 극대화

연속 배치 처리의 장점

```
# vLLM 연속 배치 처리 방식의 개념
def continuous_batch_processing(scheduler):
```

```
while scheduler.has_unfinished_seqs():
# 1. 현재 실행 중인 요청들 스케줄링
# 2. 완료된 요청들 제거
# 3. 새로운 요청들 추가
# 4. 배치 크기 동적 조정
scheduler_output = scheduler.schedule()
process_iteration(scheduler_output)
```

성능 향상 효과:

- **처리량 증가**: 기존 대비 2-4배 향상[1]
- **지연시간 감소**: 평균 응답 시간 50% 단축[2]
- **자원 활용률**: GPU 활용률 90% 이상 달성[3]

> [1] vLLM 공식 논문 "Efficient Memory Management for Large Language Model Serving with PagedAttention" 기준

> [2] A100 GPU, LLaMA-7B 모델 기준 내부 벤치마크 결과

> [3] 실제 수치는 모델 크기, 하드웨어, 워크로드에 따라 달라질 수 있음

Scheduler 클래스 구조 분석

핵심 구성 요소

vLLM의 스케줄러는 `vllm/core/scheduler.py` 에 구현되어 있으며, 다음과 같은 주요 구성 요소를 가집니다:

```
class Scheduler:
def __init__(self, scheduler_config, cache_config, lora_config, ...):
```

```
# 스케줄링 설정
self.scheduler_config = scheduler_config
self.cache_config = cache_config
# 요청 큐 관리
self.waiting: Deque[SequenceGroup] = deque() # 대기 중인 요청들
self.running: Deque[SequenceGroup] = deque() # 실행 중인 요청들
self.swapped: Deque[SequenceGroup] = deque() # 스왑된 요청들
# 블록 매니저 (메모리 관리)
self.block_manager = BlockSpaceManagerImpl(...)
# 선점 모드 설정
self.user_specified_preemption_mode = scheduler_config.preemption_mode
```

스케줄링 큐 시스템

vLLM은 3개의 주요 큐를 통해 요청 상태를 관리합니다:

1. **waiting 큐**: 새로 도착한 요청들과 재계산을 위해 선점된 요청들
2. **running 큐**: 현재 실행 중인 요청들 (prefill 및 decode 단계)
3. **swapped 큐**: 메모리 부족으로 CPU로 스왑된 요청들

SchedulingBudget을 통한 동적 배치 크기 조정

SchedulingBudget 클래스 구조

```
@dataclass
class SchedulingBudget:
```

```
"""스케줄링을 위한 사용 가능한 슬롯"""
token_budget: int # 토큰 예산
max_num_seqs: int # 최대 시퀀스 수
_num_batched_tokens: int = 0  # 현재 배치된 토큰 수
_num_curr_seqs: int = 0  # 현재 시퀀스 수
_num_cached_tokens: int = 0  # 캐시된 토큰 수
def can_schedule(self, *, num_new_tokens: int, num_new_seqs: int):
"""새로운 토큰과 시퀀스를 스케줄링할 수 있는지 확인"""
return (self.num_batched_tokens + num_new_tokens <= self.token_budget
and self.num_curr_seqs + num_new_seqs <= self.max_num_seqs)
def remaining_token_budget(self):
"""남은 토큰 예산 반환"""
return self.token_budget - self.num_batched_tokens
```

동적 예산 관리

SchedulingBudget은 다음과 같은 방식으로 동적으로 관리됩니다:

```
def add_num_batched_tokens(self, req_id: str, num_batched_tokens: int,
num_cached_tokens: int = 0):
"""배치에 토큰 추가"""
if req_id in self._request_ids_num_batched_tokens:
return # 중복 업데이트 방지
self._request_ids_num_batched_tokens.add(req_id)
```

```
self._num_batched_tokens += num_batched_tokens
self._num_cached_tokens += num_cached_tokens


def  subtract_num_batched_tokens(self, req_id: str, num_batched_tokens: int):
"""배치에서 토큰 제거"""
if req_id in self._request_ids_num_batched_tokens:
self._request_ids_num_batched_tokens.remove(req_id)
self._num_batched_tokens -= num_batched_tokens
```

예산 관리의 핵심 원칙:

- **토큰 기반 제한**: `max_num_batched_tokens` 설정값 준수
- **시퀀스 수 제한**: `max_num_seqs` 설정값 준수
- **캐시 토큰 고려**: 프리픽스 캐싱된 토큰은 "무료" 처리
- **중복 방지**: 동일 요청의 중복 업데이트 방지

PreemptionMode의 스와핑과 재계산 전략

선점 모드 정의

```
class PreemptionMode(enum.Enum):
"""선점 모드
1. Swapping: 선점된 시퀀스의 블록을 CPU 메모리로 스왑하고
재개 시 다시 GPU로 스왑인
2. Recomputation: 선점된 시퀀스의 블록을 폐기하고
```

```
재개 시 새로운 프롬프트로 취급하여 재계산
"""
SWAP = enum.auto()
RECOMPUTE = enum.auto()
```

선점 모드 결정 로직

vLLM은 다음과 같은 휴리스틱을 사용하여 선점 모드를 결정합니다:

```
def _preempt(self, seq_group: SequenceGroup,
blocks_to_swap_out: List[Tuple[int, int]]) -> PreemptionMode:
"""시퀀스 그룹을 선점하고 선점 모드를 반환"""
# 사용자 지정 모드가 없는 경우 자동 결정
if self.user_specified_preemption_mode is None:
if seq_group.get_max_num_running_seqs() == 1:
# 단일 시퀀스: 재계산 모드 (오버헤드가 낮음)
preemption_mode = PreemptionMode.RECOMPUTE
else:
# 다중 시퀀스 (빔 서치 등): 스왑 모드
preemption_mode = PreemptionMode.SWAP
else:
# 사용자 지정 모드 사용
preemption_mode = self.user_specified_preemption_mode
# 선점 실행
```

```
if preemption_mode == PreemptionMode.RECOMPUTE:
self._preempt_by_recompute(seq_group)
elif preemption_mode == PreemptionMode.SWAP:
self._preempt_by_swap(seq_group, blocks_to_swap_out)
return preemption_mode
```

재계산 기반 선점

```
def _preempt_by_recompute(self, seq_group: SequenceGroup) -> None:
"""재계산을 통한 선점"""
seqs = seq_group.get_seqs(status=SequenceStatus.RUNNING)
assert len(seqs) == 1 # 단일 시퀀스만 지원
for seq in seqs:
seq.status = SequenceStatus.WAITING # 대기 상태로 변경
self.free_seq(seq) # 메모리 해제
seq.reset_state_for_recompute() # 재계산을 위한 상태 리셋
self._free_seq_group_cross_attn_blocks(seq_group)
```

재계산 모드의 특징:

- **메모리 효율성**: 즉시 GPU 메모리 해제
- **낮은 오버헤드**: CPU-GPU 간 데이터 전송 불필요
- **단일 시퀀스 제한**: 빔 서치 등 다중 시퀀스 미지원

스왑 기반 선점

```python
def _preempt_by_swap(self, seq_group: SequenceGroup,
blocks_to_swap_out: List[Tuple[int, int]]) -> None:
"""스왑을 통한 선점"""
self._swap_out(seq_group, blocks_to_swap_out)


def _swap_out(self, seq_group: SequenceGroup,
blocks_to_swap_out: List[Tuple[int, int]]) -> None:
"""GPU에서 CPU로 스왑 아웃"""
if not self.block_manager.can_swap_out(seq_group):
raise RuntimeError("CPU 스왑 공간 부족으로 중단됨")
# 블록 매니저를 통해 스왑 실행
mapping = self.block_manager.swap_out(seq_group)
blocks_to_swap_out.extend(mapping)
# 시퀀스 상태를 SWAPPED로 변경
for seq in seq_group.get_seqs(status=SequenceStatus.RUNNING):
seq.status = SequenceStatus.SWAPPED
```

스왑 모드의 특징:

- **상태 보존**: 계산된 KV 캐시 상태 유지
- **다중 시퀀스 지원**: 빔 서치 등 복잡한 생성 방식 지원
- **높은 오버헤드**: CPU-GPU 간 데이터 전송 필요

SchedulerOutputs의 스케줄링 결과 처리

SchedulerOutputs 구조

```
@dataclass
class SchedulerOutputs:
"""스케줄러의 스케줄링 결정 결과"""
# 스케줄된 시퀀스 그룹들
scheduled_seq_groups: GenericSequence[ScheduledSequenceGroup]
# 프리필 그룹 수
num_prefill_groups: int
# 총 배치된 토큰 수
num_batched_tokens: int
# 스왑 인/아웃할 블록들
blocks_to_swap_in: List[Tuple[int, int]]
blocks_to_swap_out: List[Tuple[int, int]]
# 복사할 블록들
blocks_to_copy: List[Tuple[int, int]]
# 무시된 시퀀스 그룹들
ignored_seq_groups: List[SequenceGroup]
# 룩어헤드 슬롯 수 (추측 디코딩용)
num_lookahead_slots: int
# 실행 큐 크기
running_queue_size: int
# 선점된 요청 수
```

```
preempted: int
```

스케줄링 결과 검증

```
def __post_init__(self):
"""스케줄링 결과 검증"""
# 스왑 인과 스왑 아웃은 동시에 발생하지 않음
assert not (self.blocks_to_swap_in and self.blocks_to_swap_out)
# LoRA 요청 수 계산
self.num_loras: int = len(self.lora_requests)
if  self.num_loras > 0:
self._sort_by_lora_ids()

def  is_empty(self) -> bool:
"""스케줄링 결과가 비어있는지 확인"""
return (not self.scheduled_seq_groups and
not self.blocks_to_swap_in and
not self.blocks_to_swap_out and
not self.blocks_to_copy)
```

요청 우선순위 결정과 선점 정책

우선순위 기반 스케줄링

vLLM은 다음과 같은 우선순위 정책을 지원합니다:

```
def _get_priority(self, seq_group: SequenceGroup) -> Tuple[Optional[int], float
"""시퀀스 그룹의 우선순위 반환
사용자 정의 우선순위를 최우선으로 하고,
그 다음으로 도착 시간을 고려
"""
return seq_group.priority, seq_group.arrival_time
```

우선순위 기반 선점

```
def _schedule_priority_preemption(self, budget: SchedulingBudget) -> int:
"""우선순위 기반 강제 선점 실행"""
waiting_queue = self.waiting
running_queue = deque(sorted(self.running, key=self._get_priority))
force_preemption_count = 0
if waiting_queue:
seq_group = waiting_queue.popleft()
# 우선순위 역전이 존재하는 경우에만 선점
while (running_queue and
self._get_priority(running_queue[-1]) >
self._get_priority(seq_group)):
# 대기 중인 시퀀스가 할당 가능한지 확인
```

```
can_allocate = self.block_manager.can_allocate(seq_group)

if can_allocate == AllocStatus.OK and budget.can_schedule(...):

break

# 희생자 시퀀스 그룹 선점

victim_seq_group = running_queue.pop()

self._preempt(victim_seq_group, blocks_to_swap_out)

waiting_queue.appendleft(victim_seq_group)

force_preemption_count += 1

return force_preemption_count
```

우선순위 정책의 특징:

- **우선순위 역전 방지**: 높은 우선순위 요청이 낮은 우선순위 요청에 의해 차단되지 않음
- **공정성 보장**: 동일 우선순위 내에서는 FCFS(First Come First Served) 정책
- **동적 조정**: 실행 중에도 우선순위에 따른 선점 가능

처리량 최적화를 위한 스케줄링 전략

청크 기반 프리필 (Chunked Prefill)

vLLM은 긴 프롬프트를 청크 단위로 나누어 처리하여 지연시간을 최적화합니다:

```
def  _get_num_new_uncached_and_cached_tokens(

self, seq_group: SequenceGroup, status: SequenceStatus,

enable_chunking: bool, budget: SchedulingBudget,

partial_prefill_metadata: Optional[PartialPrefillMetadata] = None
```

```
) -> Tuple[int, int]:

"""새로운 캐시되지 않은 토큰과 캐시된 토큰 수 계산"""

if enable_chunking and len(seqs) == 1:

# 청킹이 활성화되고 단일 시퀀스인 경우

num_uncached_new_tokens = self._chunk_new_tokens_to_schedule(

self.scheduler_config,

self.cache_config,

budget,

self._get_prompt_limit(seq_group),

num_uncached_new_tokens,

self.partial_prefill_budget_lookup_list,

partial_prefill_metadata,

)

return num_uncached_new_tokens, num_cached_new_tokens
```

부분 프리필 메타데이터

```
@dataclass

class PartialPrefillMetadata:

"""부분 프리필 정보 관리"""

schedulable_prefills: int # 스케줄 가능한 프리필 수

long_prefills: int # 긴 프리필 요청 수

scheduler_config: SchedulerConfig
```

```
def can_schedule(self, seq_group: SequenceGroup) -> bool:
"""동시 부분 프리필이 활성화된 경우 긴 요청 수를 제한"""
return not (
seq_group.first_seq.get_num_new_tokens() >
self.scheduler_config.long_prefill_token_threshold and
self.long_prefills >=
self.scheduler_config.max_long_partial_prefills and
self.scheduler_config.max_num_partial_prefills > 1
)
```

지연 스케줄링 (Delay Scheduling)

```
def _passed_delay(self, now: float) -> bool:
"""지연 스케줄링 통과 여부 확인"""
if self.prev_prompt:
self.last_prompt_latency = now - self.prev_time
self.prev_time, self.prev_prompt = now, False
# 대기 큐가 채워질 수 있도록 프롬프트 스케줄링 지연
if self.scheduler_config.delay_factor > 0 and self.waiting:
earliest_arrival_time = min([e.metrics.arrival_time for e in self.waiting])
passed_delay = (
(now - earliest_arrival_time) >
(self.scheduler_config.delay_factor * self.last_prompt_latency) or
```

```
not self.running

)

else:

passed_delay = True

return passed_delay
```

성능 효과 및 벤치마크 결과

처리량 향상

vLLM의 연속 배치 처리는 다음과 같은 성능 향상을 달성합니다:

| 메트릭 | 기존 방식 | vLLM | 향상률 |

|--------|----------|------|--------|

| 처리량 (tokens/sec) | 1,200 | 4,800 | 4.0x |

| 평균 지연시간 (ms) | 2,400 | 1,200 | 2.0x |

| GPU 활용률 (%) | 65 | 92 | 1.4x |

| 동시 요청 수 | 32 | 128 | 4.0x |

> **벤치마크 환경**: NVIDIA A100 80GB, LLaMA-7B 모델, 평균 입력 길이 512토큰, 평균 출력 길이 128토큰

> **출처**: vLLM 공식 벤치마크 및 내부 성능 테스트 결과

> **주의**: 실제 성능은 모델, 하드웨어, 워크로드 특성에 따라 달라질 수 있습니다.

메모리 효율성

```
# 메모리 사용량 최적화 예시
```

```
def memory_efficiency_comparison():
"""메모리 효율성 비교"""
# 기존 방식: 고정 배치 크기
static_memory_usage = batch_size * max_seq_len * hidden_size
# vLLM 방식: 동적 배치 크기 + PagedAttention
dynamic_memory_usage = sum(
actual_seq_len * hidden_size
for actual_seq_len in current_batch_seq_lens
)
memory_savings = 1 - (dynamic_memory_usage / static_memory_usage)
return memory_savings # 일반적으로 40-60% 메모리 절약
```

실제 워크로드에서의 성능

ChatBot 워크로드:

- 평균 응답 시간: 1.8초 → 0.9초 (50% 감소)
- 동시 사용자 수: 100명 → 400명 (4배 증가)

코드 생성 워크로드:

- 처리량: 2,100 tokens/sec → 8,400 tokens/sec (4배 증가)
- 큐 대기 시간: 5.2초 → 1.3초 (75% 감소)

결론

vLLM의 배치 처리 및 스케줄링 시스템은 다음과 같은 핵심 혁신을 통해 뛰어난 성능을 달성합니다:

1. **연속 배치 처리**: 동적 배치 구성으로 자원 활용률 극대화

2. **지능적 선점**: 재계산과 스왑을 적절히 조합한 메모리 관리

3. **우선순위 스케줄링**: 공정성과 효율성을 균형있게 고려

4. **청크 기반 처리**: 긴 시퀀스의 지연시간 최적화

5. **동적 예산 관리**: 실시간 자원 할당 최적화

이러한 기술들의 시너지 효과로 vLLM은 기존 솔루션 대비 4배 이상의 처리량 향상과 50% 이상의 지연시간 단축을 달성하며, 실제 프로덕션 환경에서 안정적이고 효율적인 LLM 서빙을 가능하게 합니다.